London使用Tokyo的dns解析binance

0. 通用准备步骤（两边都可以这么做）

0.1 更新系统 & 安装工具

`sudo apt update`
`sudo apt install -y dnsmasq dnsutils traceroute mtr-tiny`

0.2 禁用 systemd-resolved（两边都建议关掉）

`sudo systemctl disable --now systemd-resolved`
`sudo rm -f /etc/resolv.conf`

> **注意：**
> 这一步一做完，系统就没有 DNS 了，所以要马上写一个新的 `/etc/resolv.conf`。

1️⃣ Tokyo 侧配置（10.0.0.2：作为“东京 DNS 服务”）

1.1 写 `/etc/resolv.conf`（给 Tokyo 自己用的）

Tokyo 上先写一个能用的系统 DNS（给 dnsmasq 的上游作为备用，不依赖它也行）：

`echo "nameserver 1.1.1.1" | sudo tee /etc/resolv.conf`

1.2 配置 dnsmasq：监听 10.0.0.2，转发到公网 DNS

新建配置文件：

`sudo tee /etc/dnsmasq.d/tokyo-dns.conf >/dev/null << 'EOF'`

Code block

```
# ===== bitstamp 走 frankfurt-1 =====
server=/bitstamp.net/10.0.2.1

# ===== 其他走本地上游 =====
server=169.254.169.253
no-resolv
```

```

listen-address=127.0.0.1,10.0.1.2, 10.0.2.2
bind-interfaces
```

重启：

`sudo systemctl restart dnsmasq`
`sudo systemctl enable dnsmasq`

1.3 Tokyo 侧防火墙允许 GRE 上的 DNS

如果你用 `ufw`（你现在是用的）：

`# 允许从 GRE 进来的 DNS 请求`
`sudo ufw allow in  on gre1 to any port 53 proto udp`
`sudo ufw allow in  on gre1 to any port 53 proto tcp`

`# 允许从 GRE 出去的 DNS 回包`
`sudo ufw allow out on gre1 to any port 53 proto udp`
`sudo ufw allow out on gre1 to any port 53 proto tcp`

1.4 Tokyo 自检：确认本机 DNS 正常

`# 确认 dnsmasq 正在监听 10.0.0.2:53`
`sudo ss -lnptu | grep :53`

`# 通过 tokyo 自己的 dnsmasq 解析 binance`
`dig +short api.binance.com @10.0.0.2`
`dig +short fapi.binance.com @10.0.0.2`
`dig +short stream.binance.com @10.0.0.2`

你应该看到类似：

`d3h36i1mno13q3.cloudfront.net.`
`13.225.169.48`

再验证 RTT 很低：

`ping -c 3 api.binance.com`
`# 0.x ms，反向名字里带 nrtXX.r.cloudfront.net，说明是东京 POP`

2️⃣ London 侧配置（127.0.0.1：Split DNS）

2.1 写 `/etc/resolv.conf`（让系统使用本机 dnsmasq）

`echo "nameserver 127.0.0.1" | sudo tee /etc/resolv.conf`

2.2 配置 dnsmasq：只对 binance 走 Tokyo，其它走 London

新建：

`sudo tee /etc/dnsmasq.d/binance-split.conf >/dev/null << 'EOF'`

Code block

```
# ===== binance 走 tokyo-2 =====
server=/binance.com/10.0.0.2
server=/binance.net/10.0.0.2

# ===== 其他走本地上游 =====
server=169.254.169.253
no-resolv

# 监听本机
listen-address=127.0.0.1,10.0.0.1
bind-interfaces
```

重启：

`sudo systemctl restart dnsmasq`
`sudo systemctl enable dnsmasq`

2.3 London 侧防火墙（一般这里 DNS 不走 gre1，只是普通出站）

通常你不需要特别为 127.0.0.1:53 配 ufw 规则，只要 **允许出站** 即可（默认 allow out）。

你已经有：

`53/udp ALLOW OUT Anywhere on gre1`
`53/tcp ALLOW OUT Anywhere on gre1`

这对后续可能直接 dig @10.0.0.2 也是有用的，可以保留。

2.4 London 自检：确认 Split DNS 生效

2.4.1 确认 dnsmasq 在本机监听

```
sudo ss -lnptu | grep :53
# 看到 127.0.0.1:53 即可
```

2.4.2 确认能直接访问 Tokyo DNS

```
dig +short api.binance.com @10.0.0.2
```

应与 Tokyo 那边结果一致。

2.4.3 确认“分流规则”生效

```
# 走本机 dnsmasq，应该等价于上面 @10.0.0.2 的结果（即东京 POP IP）
dig +short api.binance.com
dig +short fapi.binance.com
dig +short stream.binance.com

# Kraken 应该走 London，本地 DNS
dig +short ws.kraken.com
```

然后对比延迟：

```
ping -c 3 api.binance.com      # 会比在 london 原生解析稍高（多了一跳 tokyo）
ping -c 3 ws.kraken.com        # RTT 极低（伦敦直连）
```

3️⃣ 有用的检测脚本

下面给你两边各一个简单检测脚本，方便你以后一键检查。

3.1 Tokyo：`/usr/local/bin/check-tokyo-dns.sh`

```
sudo tee /usr/local/bin/check-tokyo-dns.sh >/dev/null << 'EOF'
#!/usr/bin/env bash
set -e

echo "== Tokyo dnsmasq listening =="
sudo ss -lnptu | grep ":53" || echo "!! nothing on :53"

echo
echo "== dig api.binance.com via 10.0.0.2 =="
dig +short api.binance.com @10.0.0.2 || echo "!! dig failed"

echo
```

```
echo "== ping api.binance.com (3 packets) ==" 
ping -c 3 api.binance.com || echo "!! ping failed"
EOF
sudo chmod +x /usr/local/bin/check-tokyo-dns.sh
```

使用：

```
check-tokyo-dns.sh
```

3.2 London：`/usr/local/bin/check-london-dns.sh`

```
sudo tee /usr/local/bin/check-london-dns.sh >/dev/null << 'EOF'
#!/usr/bin/env bash
set -e

echo "== dnsmasq listening on 127.0.0.1:53 ==" 
sudo ss -lnptu | grep "127.0.0.1:53" || echo "!! dnsmasq not on 127.0.0.1:53"

echo
echo "== ping Tokyo GRE IP (10.0.0.2) ==" 
ping -c 3 10.0.0.2 || echo "!! ping 10.0.0.2 failed"

echo
echo "== dig api.binance.com via Tokyo @10.0.0.2 ==" 
dig +short api.binance.com @10.0.0.2 || echo "!! dig via 10.0.0.2 failed"

echo
echo "== dig api.binance.com via local system DNS (split DNS) ==" 
dig +short api.binance.com || echo "!! dig via local DNS failed"

echo
echo "== dig ws.kraken.com via local system DNS (should be London) ==" 
dig +short ws.kraken.com || echo "!! dig ws.kraken.com failed"
EOF
sudo chmod +x /usr/local/bin/check-london-dns.sh
```

使用：

```
check-london-dns.sh
```

4️⃣ 小提示：更新 Binance 路由时记得过滤掉奇怪地址

你现在已经在路由表里把一堆日本 AWS 段都指到 `10.0.0.2` 了，以后写自动更新脚本时建议：

- 只对 `dig +short api.binance.com` / `fapi` / `stream` 的结果加路由
- 过滤掉：
  - `127.0.0.0/8`
  - `10.0.0.0/8` 等私网
- 避免再出现 `127.0.0.1 via 10.0.0.2 dev gre1` 这种坑爹路由 😂